



KOM120C **PEMROGRAMAN**

K12 | Pekan Ke-13 **Data Structure and Container**

Lanjutan: Vector, Set, and Map

Tim Pengajar Pemrograman IPB University

Learning Outcome

- Mahasiswa memahami pengertian struktur data dan container.
- Mahasiswa memahami container Vector, Set, dan Map
- Mahasiswa dapat memahami persamaan dan perbedaan penggunaan Array, List, Vector, dan Set, serta Map
- Mahasiswa mampu membuat program menggunakan Vector, Set, dan Map.

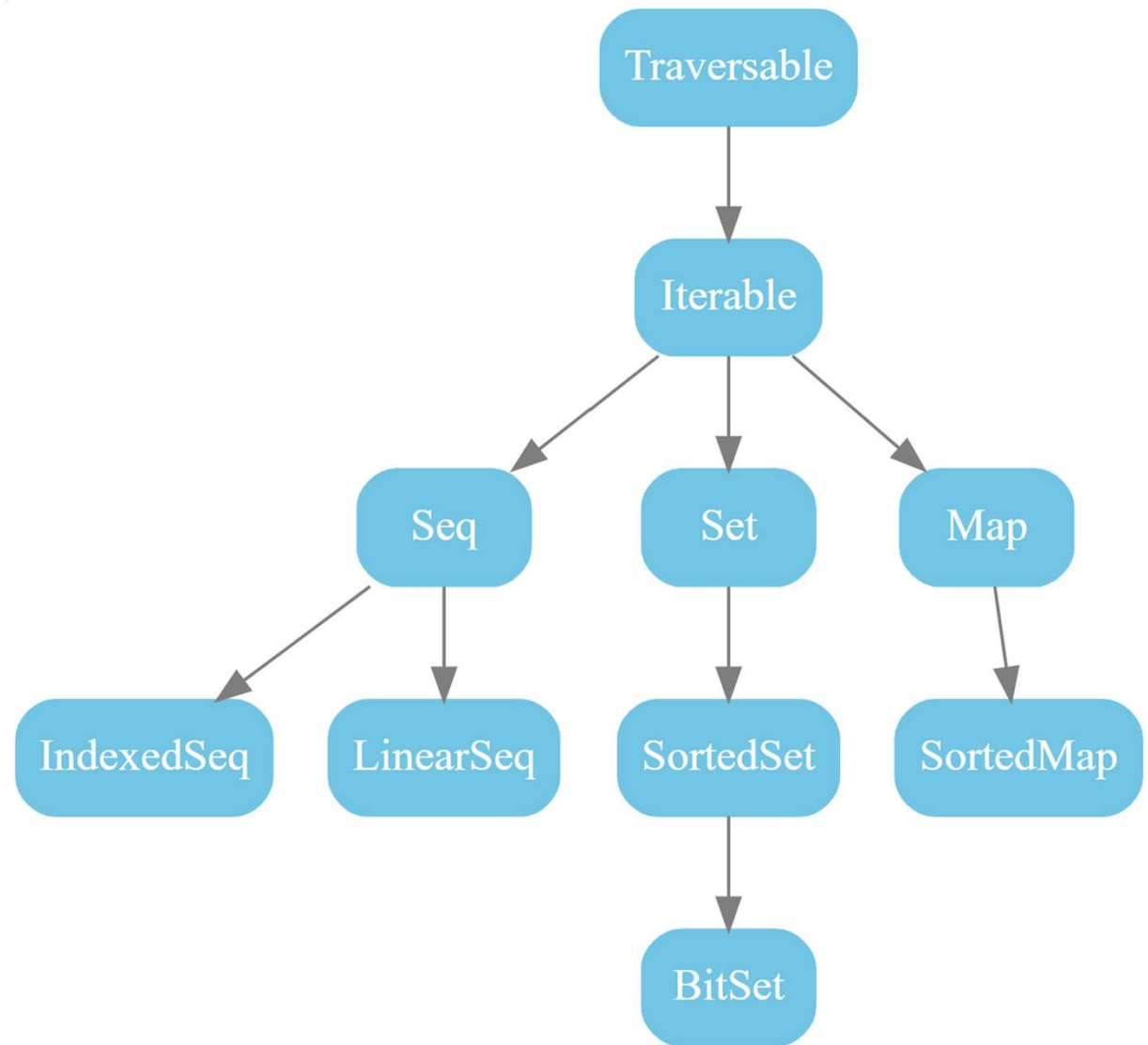


Review : Scala Collections

- Koleksi dalam Scala (**Scala Collections**) secara sistematis membedakan antara koleksi yang dapat diubah (**mutable**) dan tidak dapat diubah (**immutable**).
- Koleksi mutable dapat diubah secara langsung, sedangkan koleksi immutable tidak dapat diubah, namun kita masih memiliki operasi untuk penambahan, penghapusan, atau pembaruan menjadi koleksi yang baru, namun koleksi awal tidak berubah.
- Setiap koleksi tersimpan dalam packages:
 - `scala.collection`
 - `scala.collection.mutable`
 - `scala.collection.immutable`
- By default, Scala always picks immutable collections. Sebagai contoh, Set masuk ke dalam `scala.collection`. Sehingga, jika ingin menggunakan versi yang mutable, harus dituliskan:

```
import scala.collection.mutable
                        atau
import scala.collection.mutable.Set
```

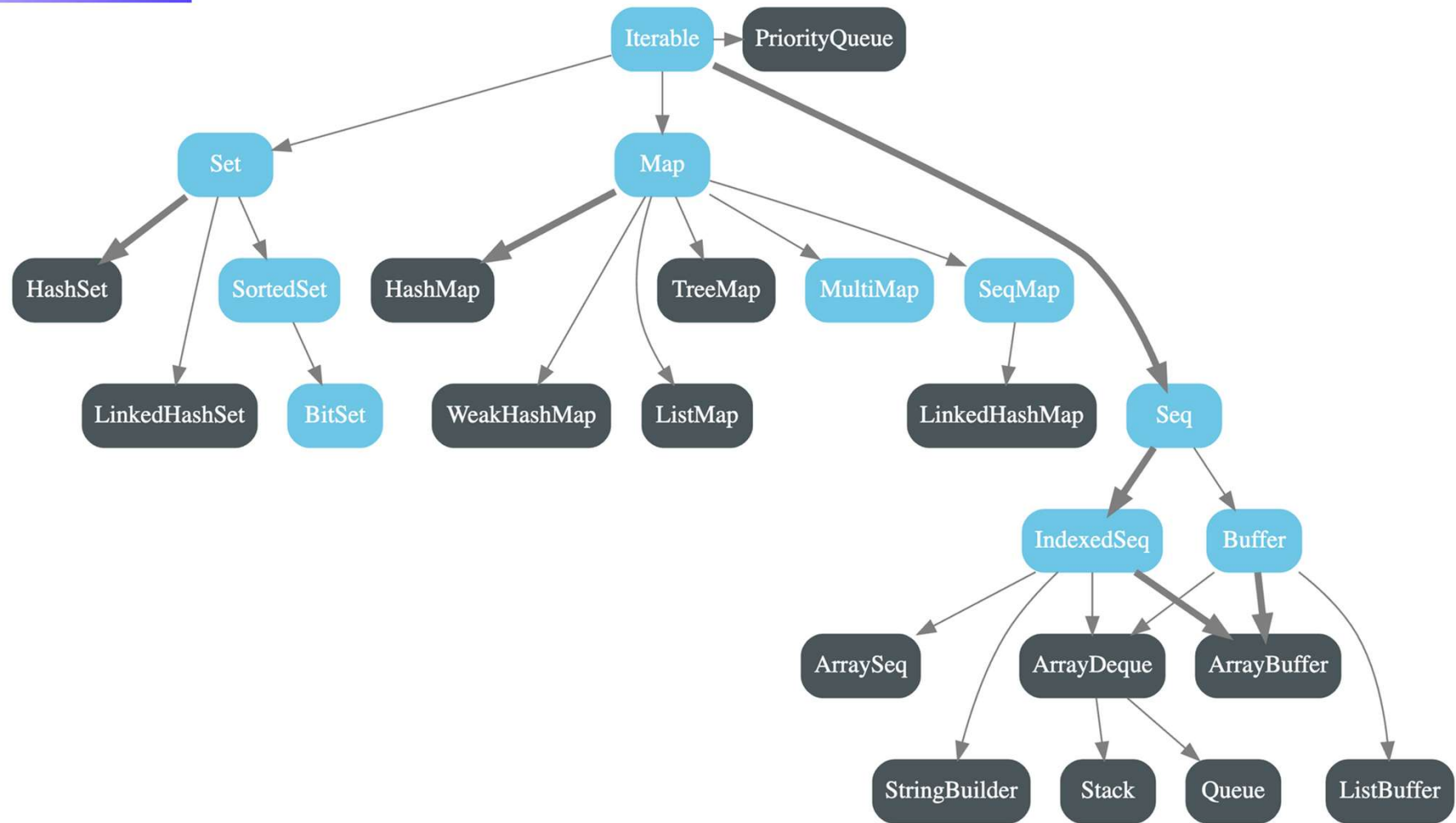
scala.collection



scala.collection.immutable



scala.collection.mutable



Vector

- Vector adalah tipe koleksi **immutable** yang menyediakan akses secara random ke setiap elemennya.
- Mirip Array atau List, setiap elemen dalam Vector juga memiliki tipe data yang sama. Vector dikenal lebih efisien khususnya ketika bekerja dengan koleksi yang besar.

```
val nums = Vector(1, 2, 3, 4, 5)
val strs = Vector("Java", "Scala", "Phyton", "C++")

println(nums)
println(strs.mkString(", "))
```



```
Vector(1, 2, 3, 4, 5)
Java, Scala, Phyton, C++
```

Mengolah Vector

- Meskipun kita tidak dapat mengubah isi elemen Vector, kita dapat melakukan operasi untuk menambah elemen Vector atau menggabungkan dua buah Vector menjadi satu.

```
val nums = Vector(3, 7, 2)
val num2 = Vector(10, 15)

val addAfter = nums :+ 9
val addBefore = 0 +: nums
val gab = nums ++ num2
println(addAfter)
println(addBefore)
println(gab)
```



```
Vector(3, 7, 2, 9)
Vector(0, 3, 7, 2)
Vector(3, 7, 2, 10, 15)
```

- Mendapatkan head dan tail dari suatu Vector

```
val nums = Vector(3, 7, 2)
println(nums.head)
println(nums.tail)
```



```
3
Vector(7, 2)
```

HOF pada Vector

- Seperti halnya tipe koleksi lainnya, HOF juga dapat diterapkan pada Vector. Misalnya .sum, .max, .reverse, dan sebagainya.

```
val nums = Vector(1, 3, 9, 7, 2)
println(nums.sum)      // jumlah elemen
println(nums.max)      // nilai terbesar
println(nums.min)      // nilai terkecil
println(nums.sum/nums.size) // rata-rata
println(nums.reverse)  // membalik elemen
println(nums.sorted)   // mengurutkan
```

```
22
9
1
4
Vector(2, 7, 9, 3, 1)
Vector(1, 2, 3, 7, 9)
```

- Kadang diperlukan pengolahan Vector menggunakan fungsi rekursif seperti contoh menjumlahkan setiap elemen Vector berikut:

```
def sum(V: Vector[Int]): Int = {
  if (V.isEmpty) 0
  else V.head+sum(V.tail)
}
```

Contoh #1

Problem

Buatlah fungsi rekursif untuk melakukan merge sort dari 2 vector yang elemennya sudah terurut, menghasilkan sebuah vector baru yang terurut.

scala

```
def merge(V1: Vector[Int], V2: Vector[Int]): Vector[Int] = {  
  if (V1.isEmpty) V2  
  else if (V2.isEmpty) V1  
  else if (V1.head < V2.head) V1.head +: merge(V1.tail, V2)  
  else V2.head +: merge(V1, V2.tail)  
}
```

```
def main(args: Array[String]): Unit = {  
  val dt1 = Vector(1, 3, 9)  
  val dt2 = Vector(4, 8)  
  println(merge(dt1, dt2))  
}
```



Vector(1, 3, 4, 8, 9)

Set

- Set adalah tipe koleksi yang merepresentasikan konsep "himpunan", dimana setiap elemen hanya mungkin muncul maksimal sekali, sehingga tidak ada elemen yang terduplikasi.
- Secara default, Set bersifat immutable (elemen tidak dapat diganti).
- Untuk Set yang bersifat mutable, terdapat versi mutable.Set yang dapat digunakan.

```
// immutable set
val immutSet = Set(5,4,3,2,1)
println(immutSet)

// mutable Set
import scala.collection.mutable
val mutSet = mutable.Set(5,4,3,2,1)
println(mutSet)
```

urutan elemen **tidak penting**



```
HashSet(5, 1, 2, 3, 4)
HashSet(1, 2, 3, 4, 5)
```

Mengolah Set

Prinsip umum: operasi perubahan elemen yang dilakukan terhadap immutable collection akan menghasilkan collection yang baru, sedangkan operasi yang dilakukan terhadap mutable collection akan langsung mengubah collection itu sendiri. Demikian pula halnya pada pengolahan Set.

```
println("Immutable Set")
val immutSet = Set(5,4,3,2,1)
val addImmutSet = immutSet + 6
val reduceImmutSet = immutSet - 5
println("Penambahan Elemen: "+addImmutSet)
println("Pengurangan Elemen: "+reduceImmutSet)
```

```
println("\nMutable Set")
import scala.collection.mutable
val mutSet = mutable.Set(5,4,3,2,1)
mutSet += 6
println("Penambahan Elemen: "+mutSet)
mutSet -= 5
println("Pengurangan Elemen: "+mutSet)
```



```
Immutable Set
Penambahan Elemen: HashSet(5, 1, 6, 2, 3, 4)
Pengurangan Elemen: HashSet(1, 2, 3, 4)
```



```
Mutable Set
Penambahan Elemen: HashSet(1, 2, 3, 4, 5, 6)
Pengurangan Elemen: HashSet(1, 2, 3, 4, 6)
```

Set dan Himpunan

Karena Set merupakan representasi dari himpunan, maka tersedia operasi pada Set seperti halnya pada himpunan, seperti gabungan (union), potongan (intersection), pengurangan (difference)

```
import scala.collection.mutable
val A=mutable.Set(1,2,3,4)
val B=mutable.Set(3,4,5,6)
println(A union B) // union
println(A|B)
println("-----")
println(A intersect B) // intersection
println(A&B)
println("-----")
println(A diff B) // difference A-B
println(A.diff(B))
println("-----")
println(A.contains(3))
println(A.contains(5))
```



```
HashSet(1, 2, 3, 4, 5, 6)
HashSet(1, 2, 3, 4, 5, 6)
-----
HashSet(3, 4)
HashSet(3, 4)
-----
HashSet(1, 2)
HashSet(1, 2)
-----
true
false
```

Transformasi List to Set

Ada beberapa cara mengubah List menjadi Set.

- Menggunakan HOF toSet.

```
val myList = List(5, 1, 2, 3, 3, 5, 1)
val mySet = myList.toSet
println(mySet)
```



```
Set(5, 1, 2, 3)
```

- Menggunakan objek Set

```
val myList = List(5, 1, 2, 3, 3, 5, 1)
val mySet = Set(myList: _*)
println(mySet)
```



```
Set(5, 1, 2, 3)
```

- Menggunakan foldLeft

```
val myList = List(5, 1, 2, 3, 3, 5, 1)
val mySet = myList.foldLeft(Set.empty[Int])(_+_)
println(mySet)
```



```
Set(5, 1, 2, 3)
```

flatMap

Mirip dengan map, namun flatMap sekaligus meratakan (to flat) setiap elemen dari koleksi.

- Perhatikan perbedaan berikut ini

```
val listLang = List("Scala Funct", "CPP OOP", "Phyton Hybrid")  
println(listLang.map(_.toLowerCase))  
println(listLang.flatMap(_.toLowerCase))
```



```
List(scala funct, cpp oop, phyton hybrid)  
List(s, c, a, l, a, , f, u, n, c, t, c, p, p, , o, o, p, p, h, y, t, o, n, , h, y, b, r, i, d)
```

- Contoh kasus: melakukan parsing terhadap kalimat menjadi list kata.

```
val listLang = List("Scala Funct", "CPP OOP", "Phyton Hybrid")  
val words = listLang.flatMap(tekst => tekst.split(" "))  
println(words)
```



```
List(Scala, Funct, CPP, OOP, Phyton, Hybrid)
```

Map

- Map adalah struktur data yang menyimpan pasangan "key" (kunci) dan "value" (nilai). Contoh: hubungan antara NIM dengan Nama Mahasiswa, huruf mutu dengan nilai mutu, dsb.

```
val mhs = Map("G123"->"Siapa Saja", "G541"->"Abdillah")
val nilaiMutu = Map("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
println(nilaiMutu)
mhs.foreach{ case (key,value)=>println(key+":"+value) }
```



```
HashMap(F -> 0, A -> 4, B -> 3, C -> 2, D -> 1)
G123:Siapa Saja
G541:Abdillah
```

- Seperti halnya Set, Map memiliki 2 jenis: mutable dan immutable. Default: immutable.

```
import scala.collection.mutable
val mutableNilai = mutable.Map("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
println(mutableNilai)
```



```
HashMap(A -> 4, B -> 3, C -> 2, D -> 1, F -> 0)
```

Mengolah Map

Ingat prinsip umum operasi perubahan elemen yang dilakukan terhadap immutable collection dan mutable collection? Apa itu?

```
println("Immutable Map")
val nilaiMutu = Map("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
val addNilai = nilaiMutu + ("AB"->3.5, "BC"->2.5, "E"->0) //tambah
val newNilai = addNilai - "F" //kurang
println(nilaiMutu)
println(addNilai)
println(newNilai)

println("-----\nMutable Map")
import scala.collection.mutable
val nilai = mutable.Map[String,Double]("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
println(nilai += ("AB"->3.5, "BC"->2.5, "E"->0))
println(nilai -= "F")
```



```
Immutable Map
HashMap(F -> 0, A -> 4, B -> 3, C -> 2, D -> 1)
HashMap(E -> 0, F -> 0, A -> 4, B -> 3, C -> 2, BC -> 2.5, AB -> 3.5, D -> 1)
HashMap(E -> 0, A -> 4, B -> 3, C -> 2, BC -> 2.5, AB -> 3.5, D -> 1)
-----
Mutable Map
HashMap(A -> 4.0, AB -> 3.5, BC -> 2.5, B -> 3.0, C -> 2.0, D -> 1.0, E -> 0.0, F -> 0.0)
HashMap(A -> 4.0, AB -> 3.5, BC -> 2.5, B -> 3.0, C -> 2.0, D -> 1.0, E -> 0.0)
```

Mengakses Elemen Map

- Akses elemen dapat dilakukan dengan dua cara: (1) **cara tidak aman**: menggunakan tanda kurung mirip array, (2) **cara aman**: menggunakan HOF **get()**.
- Cara tidak aman, bila key yang diakses/dicari tidak ada, maka akan menimbulkan runtime error. Sedangkan cara aman, akan menghasilkan nilai **None**.

```
val nilaiMutu = Map("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
println(nilaiMutu("C"))
println(nilaiMutu.get("C"))
println(nilaiMutu.get("E"))
println(nilaiMutu("E")) //runtime error
```



```
2
Some(2)
None
```

- Kita dapat juga menggunakan HOF **getOrElse()** untuk memberikan nilai alternatif bagi **get()** apabila nilai key yang dicari tidak ada

```
val nilaiMutu = Map("A"->4, "B"->3, "C"->2, "D"->1, "F"->0)
println(nilaiMutu.getOrElse("E",-1))
```



```
-1
```

HOF pada Map

- Pemetaan terhadap setiap elemen (key, value) menggunakan fungsi **map**

```
import scala.collection.mutable
val nilai = mutable.Map("a"->4, "b"->3, "c"->2)
val transNilai = nilai.map { case (key,value) => (key.toUpperCase, value+1) }
println(transNilai)
```

HashMap(A -> 5, B -> 4, C -> 3)

- Memilih (filter) elemen Map berdasarkan key atau value.

```
val nilai = mutable.Map("a"->4, "b"->3, "c"->2)
val newNilai1 = nilai.filter { case (key, value) => key>"a" }
val newNilai2 = nilai.filter { case (key, value) => value % 2 == 0 }
println(newNilai1)
println(newNilai2)
```

HashMap(b -> 3, c -> 2)
HashMap(a -> 4, c -> 2)

Diskusi Kelas #1

Buatlah sebuah fungsi `uniqueWords()` yang menerima kumpulan string yang merepresentasikan kalimat-kalimat, kemudian menghasilkan himpunan (Set) dari semua kata yang ada pada kumpulan kalimat tersebut. Hint: gunakan regular expression untuk menghilangkan punctuations.

Contoh:

```
val kalimat = List(  
  "Hello, world!",  
  "Hello Scala.",  
  "Functional Programming in Scala",  
  "Scala is fun."  
)  
println(uniqueWords(kalimat))
```



```
HashSet(in, is, world, programming, scala, fun, hello, functional)
```

Diskusi Kelas #2

Buatlah sebuah fungsi yang menerima kumpulan string yang merepresentasikan kalimat-kalimat, kemudian menghasilkan himpunan Map yang menunjukkan frekuensi kemunculan dari semua kata (dalam huruf kecil) yang ada pada kumpulan kalimat tersebut.

Contoh:

```
val kalimat = List(  
  "Hello, world!",  
  "Hello Scala.",  
  "Functional Programming in Scala",  
  "Scala is fun."  
)  
println(wordFreq(kalimat))
```



```
HashMap(in -> 1, is -> 1, world -> 1, programming -> 1, scala -> 3, fun -> 1, hello -> 2, functional -> 1)
```